- Zoho Catalyst Appsail Development & Deployment Guide
  - Table of Contents
  - Project Overview
    - Key Features
  - Prerequisites
    - Required Software
    - Required Accounts
    - Required Knowledge
  - Project Structure
    - File Descriptions
      - catalyst.json
      - appsail-python/app.py
      - appsail-python/main.py
      - appsail-python/app-config.json
      - appsail-python/requirements.txt
  - Development Setup
    - 1. Install Zoho Catalyst CLI
    - 2. Login to Zoho Catalyst
    - 3. Initialize Project (if starting fresh)
    - 4. Clone/Setup Existing Project
  - Code Architecture
    - Flask Application Structure
      - Entry Point (app.py)
      - Main Application (main.py)
  - Database Configuration
    - Zoho Catalyst NoSQL Tables
      - Table 1: webhook_logs (ID: 6751000000044786)
      - Table 2: delivered_logs (ID: 6751000000048519)
    - Database Connection
  - Development Lifecycle
    - 1. Local Development
      - Setup Local Environment
      - Local Testing
    - 2. Code Development Process
      - Making Changes
      - Code Structure Best Practices
    - 3. Version Control

- Deployment Process
  - 1. Pre-Deployment Checklist
  - 2. Deploy to Appsail
    - Deploy Command
    - Deployment Process
  - 3. Post-Deployment Verification
    - Check Deployment Status
    - Test Application Endpoints
- Testing & Validation
  - 1. Health Check Tests
    - Basic Health Check
    - Database Connection Test
  - 2. Webhook Testing
    - Test Webhook Data
  - 3. Data Retrieval Test
    - Authenticated Data Access
- Troubleshooting
  - Common Issues
    - 1. Deployment Failures
    - 2. Application Not Starting
    - 3. Database Connection Issues
    - 4. Webhook Not Receiving Data
  - Debugging Commands
    - View Application Logs
    - Test Database Connection
    - Verify Application Health
- Best Practices
  - 1. Code Organization
  - 2. Database Management
  - 3. Deployment
  - 4. Security
  - 5. Monitoring
- Project URLs
  - Development Environment
  - Database Tables
- Quick Reference Commands
  - Development
  - Deployment

- Testing

Zoho Catalyst Appsail Development & Deployment Guide

Table of Contents

1. Project Overview
2. Prerequisites
3. Project Structure
4. Development Setup
5. Code Architecture
6. Database Configuration
7. Development Lifecycle
8. Deployment Process
9. Testing & Validation
10. Troubleshooting
11. Best Practices

Project Overview

This project is a **WhatsApp Webhook Handler** deployed on Zoho Catalyst Appsail that:

- Receives WhatsApp webhook data via HTTP POST requests
- Processes and stores webhook data in Zoho Catalyst NoSQL database
- Provides API endpoints for data retrieval and health monitoring
- Handles authentication for secure data access

Key Features

- Real-time webhook data collection
- Dual database table storage (webhook_logs & delivered_logs)
- RESTful API endpoints

- Health monitoring and status checks
- Error handling and logging

Prerequisites

Required Software

- **Python 3.9+** (Zoho Catalyst Appsail supports Python 3.9)
- **Zoho Catalyst CLI** - Install via npm: `npm install -g @zoho/catalyst-cli`
- **Git** for version control
- **PowerShell** (Windows) or **Terminal** (Mac/Linux)

Required Accounts

- **Zoho Developer Account** with Catalyst access
- **WhatsApp Business API** account (for webhook data)

Required Knowledge

- Python Flask framework
- RESTful API development
- JSON data handling
- Zoho Catalyst NoSQL database concepts
- Webhook integration patterns

Project Structure

```
catalyst/
├── catalyst.json                 # Project configuration
└── appsail-python/              # Main application directory
    ├── app.py                   # Entry point (Flask app wrapper)
    ├── main.py                  # Core application logic
    ├── app-config.json          # Appsail deployment configuration
    └── requirements.txt         # Python dependencies
```

File Descriptions

`catalyst.json`

```
{
  "appsail": [
    {
      "source": "appsail-python",
      "name": "test-knight"
    }
  ]
}
```

- **Purpose**: Defines the Appsail application configuration
- **source**: Directory containing the application code
- **name**: Application name on Zoho Catalyst

`appsail-python/app.py`

```
from main import app

# This file is just a wrapper around main.py
# Needed for Zoho Catalyst AppSail to properly run the application
```

- **Purpose**: Entry point for Zoho Catalyst Appsail
- **Role**: Imports the Flask app from main.py
- **Why needed**: Appsail expects app.py as the default entry point

`appsail-python/main.py`

- **Purpose**: Core application logic
- **Contains**: Flask routes, database operations, webhook processing
- **Key Components**:
  - Flask app initialization
  - Webhook endpoint (`/webhook`)
  - Health check endpoint (`/health`)
  - Database connection functions
  - Authentication logic

appsail-python/app-config.json

```
{
  "command": "sh -c \"python3 -m pip install --no-cache-dir -r requirements.txt &&
python3 -m gunicorn -w 2 -k gthread -t 120 -b 0.0.0.0:$X_ZOHO_CATALYST_LISTEN_PORT
main:app\"",
  "build_path": "./",
  "stack": "python_3_9",
  "env_variables": {},
  "memory": 256
}
```

- **Purpose**: Appsail deployment configuration
- **command**: Build and run command for the application
- **stack**: Python version (3.9)
- **memory**: Allocated memory (256MB)

appsail-python/requirements.txt

```
Flask==3.0.3
gunicorn==22.0.0
Werkzeug==3.0.3
zcatalyst-sdk
```

- **Purpose**: Python dependencies
- **Flask**: Web framework
- **gunicorn**: WSGI server for production
- **zcatalyst-sdk**: Zoho Catalyst SDK for database operations

Development Setup

1. Install Zoho Catalyst CLI

```
npm install -g @zoho/catalyst-cli
```

2. Login to Zoho Catalyst

```
catalyst login
```

- Enter your Zoho credentials
- Select the appropriate organization

3. Initialize Project (if starting fresh)

```
catalyst init
```

- Follow the prompts to create a new Catalyst project
- Select "Appsail" as the service type

4. Clone/Setup Existing Project

```
# Navigate to your project directory
cd /path/to/your/catalyst/project

# Verify project configuration
catalyst project:list
```

Code Architecture

Flask Application Structure

Entry Point (`app.py`)

```
from main import app
# Simple wrapper that imports the main Flask application
```

Main Application (`main.py`)

Key Components:

1. **Flask App Initialization**

```
app = Flask(__name__)
```

2. **Database Configuration**

```
# NoSQL Table names with correct IDs
WEBHOOK_LOGS_TABLE = "webhook_logs"  # Table ID: 6751000000044786
DELIVERED_LOGS_TABLE = "delivered_logs"  # Table ID: 6751000000048519
PROJECT_ID = "6751000000041001"
```

3. **Catalyst SDK Integration**

```
def get_catalyst_app():
    """Get Catalyst app instance with fallback mechanisms"""
    # Multiple initialization attempts for robustness
```

4. **Webhook Processing**

```
@app.route("/webhook", methods=["POST"])
def webhook():
    """Main webhook endpoint for receiving WhatsApp data"""
    # Processes incoming webhook data
    # Stores in appropriate database tables
    # Returns success response
```

5. **API Endpoints**

- `GET /` - Basic health check
- `GET /health` - Detailed health status
- `POST /webhook` - Webhook data receiver
- `GET /webhook/status` - Database connection status
- `POST /webhook/master` - Retrieve stored data (with auth)

Database Configuration

Zoho Catalyst NoSQL Tables

Table 1: webhook_logs (ID: 6751000000044786)

Purpose: Stores all incoming webhook data **Schema**:

- `webhook_id`: Unique message ID
- `status`: Message status (accepted, failed, sent, delivered)
- `recipient_id`: WhatsApp recipient ID
- `webhook_data_json`: Complete webhook payload
- `web_timestamp`: Original timestamp from webhook
- `created_time`: Record creation timestamp
- `Queue_Id`: WhatsApp queue ID
- `web_date`: Extracted date from timestamp
- `raw_webhook_data_json`: Raw webhook data (optional)

Table 2: delivered_logs (ID: 6751000000048519)

Purpose: Stores processed webhook data for delivered messages **Schema**: Same as webhook_logs

Database Connection

```
def store_webhook_data_nosql(webhook_data, table_name, raw_webhook_data=None):
    """Store webhook data in Catalyst Data Store"""
    # Handles table detection and data insertion
    # Includes error handling and logging
```

Development Lifecycle

1. Local Development

Setup Local Environment

```
# Navigate to project directory
cd appsail-python

# Install dependencies locally
pip install -r requirements.txt
```

```
# Run locally for testing
python main.py
```

Local Testing

```
# Test health endpoint
curl http://localhost:9000/health

# Test webhook endpoint
curl -X POST http://localhost:9000/webhook \
  -H "Content-Type: application/json" \
  -d '{"test": "data"}'
```

2. Code Development Process

Making Changes

1. **Edit Code**: Modify `main.py` for application logic
2. **Test Locally**: Run `python main.py` to test changes
3. **Update Dependencies**: Modify `requirements.txt` if needed
4. **Test Deployment**: Deploy to Appsail for testing

Code Structure Best Practices

- Keep `app.py` minimal (just imports)
- Put all logic in `main.py`
- Use proper error handling
- Add logging for debugging
- Follow Flask best practices

3. Version Control

```
# Initialize git repository
git init

# Add files
git add .

# Commit changes
```

```
git commit -m "Initial webhook handler implementation"

# Push to remote
git push origin main
```

Deployment Process

1. Pre-Deployment Checklist

- ☐ Code tested locally
- ☐ Dependencies updated in `requirements.txt`
- ☐ Database table IDs verified
- ☐ Environment variables configured
- ☐ Error handling implemented

2. Deploy to Appsail

Deploy Command

```
# Navigate to project root (where catalyst.json is located)
cd /path/to/catalyst/project

# Deploy the application
catalyst deploy appsail
```

Deployment Process

1. **Build Phase**:
    - Installs Python dependencies
    - Prepares application bundle
    - Validates configuration
2. **Deploy Phase**:
    - Uploads code to Zoho Catalyst
    - Starts the application
    - Configures networking

3. **Verification**:
    - Application becomes available at provided URL
    - Health checks are performed

3. Post-Deployment Verification

Check Deployment Status

```
# Check deployment status
catalyst status appsail

# View application logs
catalyst logs appsail
```

Test Application Endpoints

```
# Test health endpoint
curl https://your-app-url.catalystappsail.in/health

# Test webhook endpoint
curl -X POST https://your-app-url.catalystappsail.in/webhook \
  -H "Content-Type: application/json" \
  -d '{"test": "webhook_data"}'
```

Testing & Validation

1. Health Check Tests

Basic Health Check

```
curl https://test-knight-50030314031.development.catalystappsail.in/health
```

Expected Response: `{"status": "ok", "database": "connected"}`

Database Connection Test

```
curl https://test-knight-50030314031.development.catalystappsail.in/webhook/status
```

Expected Response: Database connection status

2. Webhook Testing

Test Webhook Data

```
# PowerShell example
$testData = @{
    object = "whatsapp_business_account"
    entry = @(@{
        id = "374947405711015"
        changes = @(@{
            value = @{
                messaging_product = "whatsapp"
                metadata = @{
                    display_phone_number = "919154699658"
                    phone_number_id = "408694838996380"
                }
                statuses = @(@{
                    id = "test_message_$(Get-Date -UFormat %s)"
                    status = "delivered"
                    timestamp = "$(Get-Date -UFormat %s)"
                    recipient_id = "917068482741"
                })
            }
            field = "messages"
        })
    })
}

$jsonData = $testData | ConvertTo-Json -Depth 10
$response = Invoke-WebRequest -Uri "https://your-app-
url.catalystappsail.in/webhook" -Method POST -Body $jsonData -ContentType
"application/json"
```

3. Data Retrieval Test

Authenticated Data Access

```
# PowerShell example
$authData = @{
    username = "your-username"
```

```
    password = "your-password"
}
$jsonAuthData = $authData | ConvertTo-Json

$response = Invoke-WebRequest -Uri "https://your-app-
url.catalystappsail.in/webhook/master" -Method POST -Body $jsonAuthData -
ContentType "application/json"
```

Troubleshooting

Common Issues

1. Deployment Failures

Error: `Project info is not present` **Solution**:

```
# Navigate to correct directory
cd /path/to/catalyst/project
# Verify catalyst.json exists
ls catalyst.json
```

Error: `ProjectId is needed` **Solution**:

```
# Set project explicitly
catalyst project:use <project-id>
```

2. Application Not Starting

Error: Internal server error **Solutions**:

- Check `requirements.txt` for missing dependencies
- Verify Python version compatibility
- Check `app-config.json` command syntax
- Review application logs

3. Database Connection Issues

Error: Catalyst app not available **Solutions**:

- Verify table IDs are correct
- Check PROJECT_ID configuration
- Ensure proper SDK initialization
- Test with fallback mechanisms

4. Webhook Not Receiving Data

Error: 404 or 500 errors on webhook endpoint **Solutions**:

- Verify endpoint URL is correct
- Check webhook payload format
- Ensure proper Content-Type headers
- Review error logs

Debugging Commands

View Application Logs

```
# View recent logs
catalyst logs appsail

# View logs with timestamps
catalyst logs appsail --timestamps

# Follow logs in real-time
catalyst logs appsail --follow
```

Test Database Connection

```
# Test database connectivity
curl https://your-app-url.catalystappsail.in/webhook/status
```

Verify Application Health

```
# Check application status
curl https://your-app-url.catalystappsail.in/health
```

Best Practices

1. Code Organization

- Keep `app.py` minimal (entry point only)
- Put all logic in `main.py`
- Use proper error handling
- Add comprehensive logging
- Follow PEP 8 style guidelines

2. Database Management

- Use proper table IDs from production
- Implement fallback mechanisms
- Handle connection failures gracefully
- Log all database operations
- Validate data before storage

3. Deployment

- Test locally before deploying
- Use version control
- Keep deployment configuration simple
- Monitor application logs
- Have rollback plan ready

4. Security

- Implement proper authentication
- Validate all incoming data
- Use HTTPS endpoints
- Store sensitive data securely
- Regular security updates

5. Monitoring

- Implement health checks
- Monitor application logs
- Set up alerts for failures
- Track webhook success rates
- Monitor database performance

Project URLs

Development Environment

- **Application URL**: `https://test-knight-50030314031.development.catalystappsail.in`
- **Health Check**: `https://test-knight-50030314031.development.catalystappsail.in/health`
- **Webhook Endpoint**: `https://test-knight-50030314031.development.catalystappsail.in/webhook`

Database Tables

- **webhook_logs**: Table ID `6751000000044786`
- **delivered_logs**: Table ID `6751000000048519`

Quick Reference Commands

Development

```
# Run locally
python main.py

# Install dependencies
pip install -r requirements.txt
```

```
# Test health
curl http://localhost:9000/health
```

Deployment

```
# Deploy to Appsail
catalyst deploy appsail

# Check status
catalyst status appsail

# View logs
catalyst logs appsail
```

Testing

```
# Health check
curl https://your-app-url.catalystappsail.in/health

# Webhook test
curl -X POST https://your-app-url.catalystappsail.in/webhook -H "Content-Type:
application/json" -d '{"test": "data"}'
```

This guide provides a complete development and deployment lifecycle for your Zoho Catalyst Appsail webhook handler project. Follow these steps to develop, deploy, and maintain your application effectively.